Практическая работа №6

Обработка ошибок и защита соединений

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Защитить операции с базой данных конструкцией try/except/finally, научиться откатывать транзакцию rollback() и гарантированно закрывать соединение, проверить поведение приложения при ошибке.

Технологии: Python 3 (try/except/finally); sqlite3 (commit, rollback, sqlite3.Error, IntegrityError); Flask; Jinja2.

Реализация: Соединение открывается до блока try, чтобы finally всегда мог его закрыть. В try выполняются execute и commit; в except sqlite3.Error — rollback(), печать «[DB ERROR] …» и возврат страницы с сообщением «Не удалось сохранить задачу». В finally — conn.close(). Для проверки в INSERT временно передавалось значение None, что вызывает ошибку NOT NULL constraint failed.

Основные модули:

- `src/app.py` — fetch_all_tasks() и ветка POST обёрнуты в try/except/finally; при ошибке выполняется rollback(), ошибка выводится в консоль, пользователь видит понятное сообщение.
- `src/templates/index.html` — шаблон из работы №5 (вывод {{ error }}).

Ход работы

Шаги 1–4. Обработка ошибок через try/except/finally

```python
import sqlite3
from flask import Flask, render_template, request, redirect, url_for
app = Flask(__name__)
DB_PATH = 'app.db'
def get_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn
def fetch_all_tasks():
    """Чтение тоже защищаем: при ошибке вернём пустой список, соединение закроется всегда."""
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT id, title, status FROM tasks ORDER BY id")
        return cursor.fetchall()
    except sqlite3.Error as e:
        print(f"[DB ERROR] {e}")
        return []
    finally:
        conn.close()
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()

        if not title:
            return render_template(
                'index.html',
                items=fetch_all_tasks(),
                error='Название задачи не может быть пустым.'
            )
        conn = get_connection()
        try:
            cursor = conn.cursor()
            cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
            conn.commit()
        except sqlite3.Error as e:
            conn.rollback()
            print(f"[DB ERROR] {e}")
            return render_template(
                'index.html',
                items=fetch_all_tasks(),
                error='Не удалось сохранить задачу. Попробуйте ещё раз.'
            )
        finally:
            conn.close()
        return redirect(url_for('index'))
    return render_template('index.html', items=fetch_all_tasks())
```

Рисунок 1 — app.py: чтение и добавление задач обёрнуты в try / except / finally, при ошибке — rollback()



Рисунок 2 — защищённое чтение: таблицы ещё нет — ошибка перехвачена ([DB ERROR] no such table), сервер продолжает работать и отдаёт страницу (200)

Шаг 5. Эксперимент 1 — ошибка NOT NULL

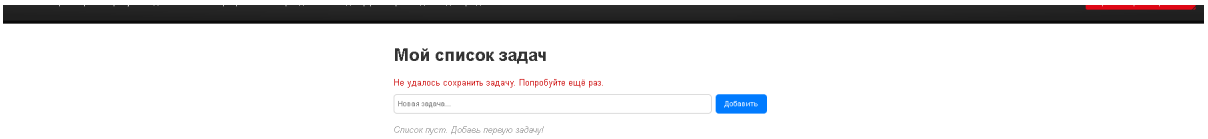


Рисунок 3 — вставка None вместо названия: пользователь видит сообщение «Не удалось сохранить задачу», лишняя запись в базу не попала

Шаг 5. Эксперимент 1 — консоль сервера

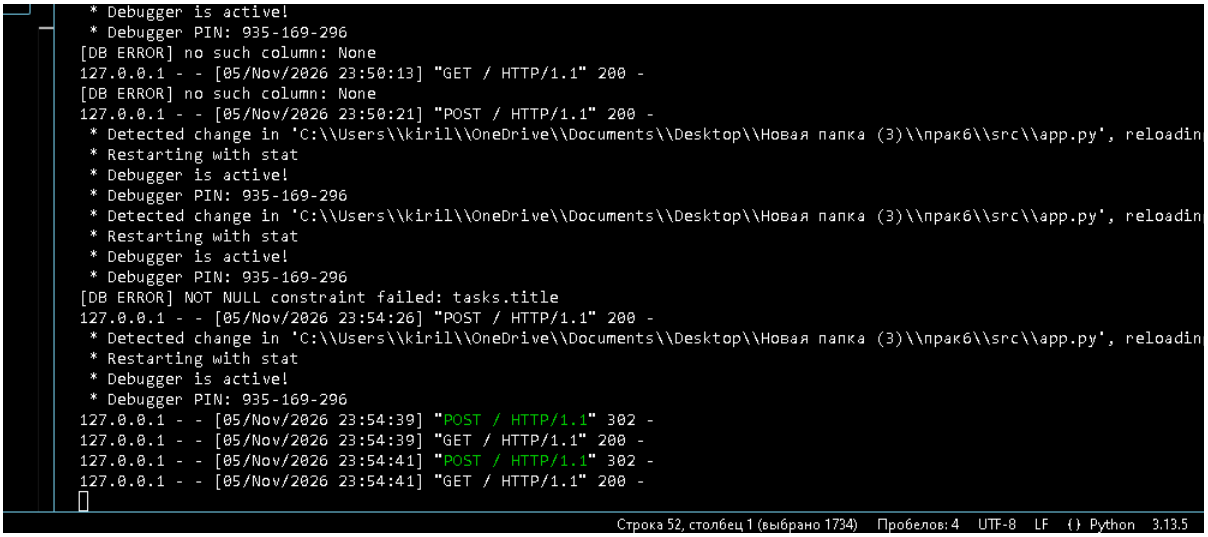


Рисунок 4 — консоль сервера: [DB ERROR] NOT NULL constraint failed: tasks.title; после возврата (title,) — POST 302 и GET 200

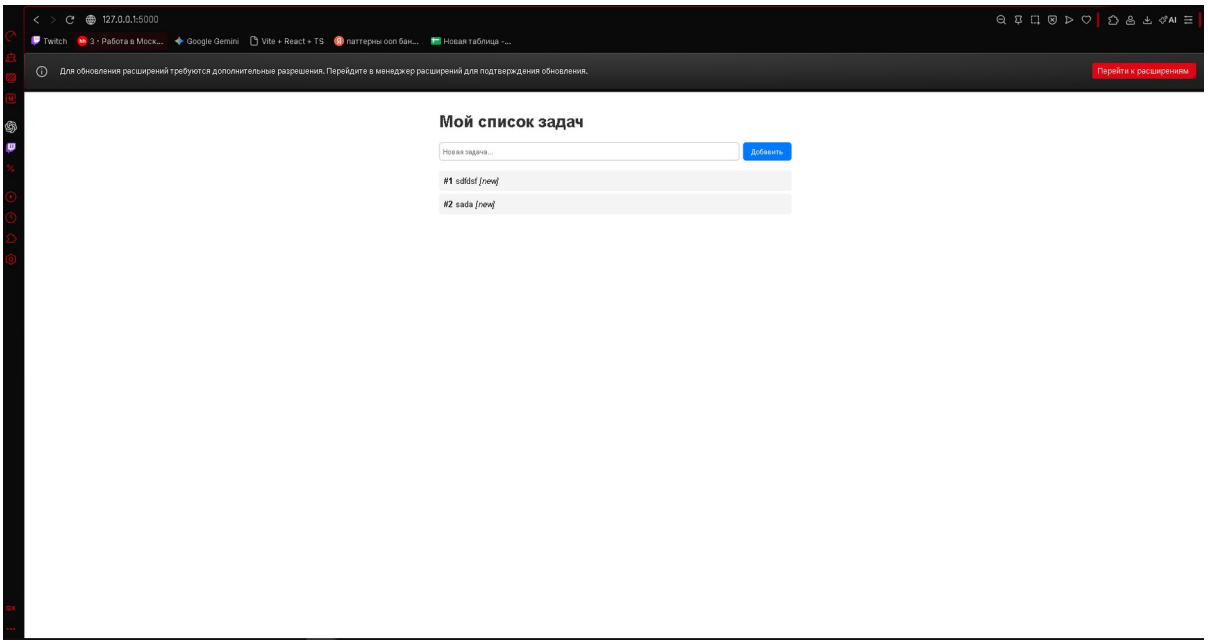


Рисунок 5 — после исправления кода сервер не падал, задачи снова добавляются

Примеры запросов и ответов

```
Эксперимент: (title,) заменено на (None,)
POST /  title=x → 200 OK, «Не удалось сохранить задачу. Попробуйте ещё раз.»
Консоль сервера: [DB ERROR] NOT NULL constraint failed: tasks.title
Количество задач в базе до и после ошибки одинаковое — rollback сработал
```

Ответы на контрольные вопросы

1. **Почему conn.close() нужно помещать в finally?**

Если в execute или commit произойдёт исключение, код после них не выполнится и соединение останется открытым. finally выполняется всегда — и при успехе, и при ошибке, и при return.

2. **Что делает conn.rollback() и когда вызывается?**

Отменяет все незафиксированные изменения текущей транзакции. Вызывается в блоке except, когда операция с базой завершилась ошибкой.

3. **Почему conn нужно открывать до try?**

Иначе при ошибке на самом открытии переменная conn не будет создана и в finally возникнет UnboundLocalError.

4. **Чем except sqlite3.Error лучше голого except:?**

Ловится только ожидаемый тип ошибок базы данных. Голый except перехватит всё подряд, включая Ctrl+C (KeyboardInterrupt) и ошибки в логике программы, и скроет их.

5. **Что будет с данными, если execute прошёл, а commit упал?**

Изменения не зафиксируются; rollback() отменит их, и база останется в исходном состоянии.

6. **Зачем показывать пользователю понятное сообщение вместо стектрейса?**

Стектрейс непонятен пользователю и раскрывает внутреннее устройство приложения (угроза безопасности). Понятное сообщение сообщает о проблеме, а сервер продолжает работать.

7. **Как проверить, что rollback откатил транзакцию?**

Сравнить количество задач до и после ошибки (в браузере, через crud_console.py или DB Browser): лишних записей не появилось.

Вывод

Все операции с базой защищены конструкцией try/except/finally с откатом транзакции. Экспериментально подтверждено, что при ошибке приложение не падает, база не блокируется и в ней не остаётся «мусорных» записей.